

Mark Frost

COIN 496 Senior Portfolio

Friday April 1, 2011

Markowitz Frost

5 Holliday Court • Columbia, South Carolina 29223
(803) 260-7535 • mafrost@csustudent.net

Objective

Searching for a position within the IT field geared toward network administration and security

Education

B.A. in Computer Science with emphasis in Math, Expected graduation, May 2012
Charleston Southern University, Charleston, SC

Skills/Qualifications

- Proficient in troubleshooting Windows and Linux based PCs
- Talented in all Windows-based software and Adobe Photoshop
- Excel in algorithm development using C++, Java, Visual Basic, and LISP
- Ability to communicate information effectively and clearly to individuals and groups
- Strong analytical and problem solving skills
- Understanding of public transportation contracting process and execution
- Knowledgeable in the operation of state and federal government agencies

Work Experience

Charleston Southern University, North Charleston, SC
Student Career Counselor, The Career Center

August 2007 – Present

- Provide technical support to staff members and install new hardware and software
- Assist students, alumni, faculty and recruiters with career center resources
- Develop and distribute marketing materials for various workshops and events
- Organize, scan, file, and fax confidential academic and employment information
- Participate in the presentation of recruitment activities and career development workshops

Federal Transit Administration, Washington, D.C.
Contract Specialist Intern, Office of Administration

June 2010 – August 2010

- Participated in a government engineer contracting colloquium with engineers from across different regions
- Gained knowledge on the process behind closeout of contracts at the end of the period
- Performed the process of archiving a contract once the closeout process has been finished
- Learned the steps behind how a Purchase Order or Request for Proposal leads to obligation of funds to the contractee

District Department of Transportation, Washington, D.C.
Intern (Mass Transit Administration)

June 2009 – August 2009

- Learned about contract writing while assisting in the revision of a Request for Proposal
- Discovered the financial aspect of acquiring vehicles for transit purposes
- Experienced the planning and execution process of DDOT's transit projects

Circuit City, Columbia, SC
Firedog Car installer

May 2007 - February 2009

- Provided quality in-car audio, alarm, and other installations with speed and accuracy
- Assisted customers with car audio equipment selection and payments

Activities

- Member of Campus Crusade for Christ, Charleston Southern University, 2009-present
- Member, Association for Computing Machinery, CSU Chapter, 2008- present
- Custom wiring for automotive aftermarket products (e.g. Alarms, LEDs, headlights), 2008-present
- Building and fixing hardware issues for PCs and Laptops, 2007-present

References available upon request

Anagram Maker

COINS 315

Data Structures

Grade: 70/100

Language written: C++

Compiler: Microsoft Visual Studio 2008

How to run: When calling the executable, there are 3 command line arguments:
Project5.exe [dictionary file] [input string] [print ("yes" or "no")]

testAnagramMaker.cpp

```
//Markowitz Frost
//Anagram Maker
//Created November 24, 2010
//Last Modified: November 30, 2010
//COIN Course: COINS 315
//Grade Received: 70/100 (incomplete)
//Comments regarding design:
/*
Basic setup of my design is to make the anagram maker as efficient as
possible, outside of reading in the dictionary. As for storing the letters
from the input string, I used absolute locations along with a count in
a single array. Meaning the letter a equals spot zero in the array, b equals
spot 1, c equals spot 2 and so on. This cut back on the number of loops
required to check if input string had the letters of the current dictionary
word being checked. Used forward slashes to mark the words used in the
dictionary.... there's a much better way to go about keeping track of
what words have been used in the dictionary.
*/

#include "AnagramMaker.h"
#include "StopWatch.h"
#include "fstream"
#include <iostream>

using namespace std;

void main(string dict, string input, string p){
    bool print=false;
    if (p == "yes") print=true;

    CStopWatch st;
    st.startTimer();
    //AnagramMaker am(dict);
    AnagramMaker am("dict1.txt");
    //st.stopTimer();
    //cout<<st.getElapsedTime()<< "seconds" <<endl;
    //system("pause");
    //st.startTimer();
    //am.findAnagrams(input, print);
    am.findAnagrams("barbara bush", true);
    st.stopTimer();
    cout<<st.getElapsedTime()<< "seconds" <<endl;
}
```

AnagramMaker.h

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>

using namespace std;

class AnagramMaker{
public:
    AnagramMaker(const string &dictionaryFile);
    ~AnagramMaker();
    void organizeDictionary();
    int findAnagrams(string input, bool print); //organizes the input
    before going to recursive function
    void StepPrint();//HERE FOR TESTING PURPOSES

private:
    struct lettersStruct{
        char letter;
        int count;
        int length; //easier to find length once and store it, only
        stored in first array spot.
    }; /*When input organized, places the number of letters available,
    could make it easier to keep track of how many are left available*/

    string subtractString(const string &lhs, lettersStruct rhs[]); //used
    for soFar when recursing back through dictionary
    void subtractArray(const string &lhs, lettersStruct rhs[]) const;
    //subtracts string from input array
    bool compare(const string &lhs, lettersStruct rhs[]) const; //see's if
    the string is in the input arr
    int findAnagrams (string soFar, lettersStruct input[], bool print);
    void organizeCurWord(string &cur); //put the word into 'temp' struct
    when called
    void resetEnd(lettersStruct input[]); //resets the end of dictionary to
    be read
    void resetDict(vector<string>::iterator start); //takes away markers of
    used words in dictionary

    char start, end; //keep track of where to start and end in
    dictionary
    //vector<temp> orgInput; //organized input with letter counts, may be
    used, would need a different values input in the findAnagrams function.
    vector<string> dictionary, orgDict; //when adding words it could get
    too big b/c of doubling in size once limit is reached.
    lettersStruct curWordArr[26], inputArr[26];
    vector<string>::iterator dictEnd, dictBegin;//, dictCur;
    string *dictionaryPoint;
};
```

AnagramMaker.cpp

```
#include "AnagramMaker.h"

AnagramMaker::AnagramMaker(const string &dictionaryFile){
    for (int i=0; i<26; i++){
        inputArr[i].count = curWordArr[i].count = 0;
        inputArr[i].length = curWordArr[i].length = 0;
        inputArr[i].letter = curWordArr[i].letter = ('a'+i); //ended up
being unnecessary
    }
    ifstream input((char*)dictionaryFile.c_str());
    if ( ! input.is_open()) { //make sure that file was found
        cout<< "Can't open file!"<<endl;
        return;
    }

    string line;
    while(getline(input, line)) {
        //for (unsigned i=0; i<line.length(); i++){
        //    for (unsigned j=i; j<line.length(); j++){
        //        if (line[j]<line[i]){
        //            char temp = line[i];
        //            line[i] = line[j];
        //            line[j] = temp;
        //        } //end if
        //    } //end for
        //} //end for
        dictionary.push_back(line); //push words from .txt file into
dictionary vector
        //organizeDictionary();
    }
    dictEnd = dictionary.end(); //pointer to dictionary end
    input.close();
    dictBegin = dictionary.begin();
}

AnagramMaker::~AnagramMaker(){

}

void AnagramMaker::organizeDictionary(){ //Not used b/c it's not efficient
    for (int i=0; i!=dictionary.size(); i++){ //organize by size of word
        for (int j=0; j!=dictionary.size(); j++){
            if (dictionary[i].length() < dictionary[j].length()){
                string temp = dictionary[i];
                dictionary[i] = dictionary[j];
                dictionary[j] = temp;
            } //end if
        } //end for
    } //end for

    /*for (int j=0; j!=dictionary.size(); j++){ //organize back into abc
order???
        if (dictionary[i][0].length() < dictionary[j][0].length()){
            } //end if*/
}
```

AnagramMaker.cpp

```

int AnagramMaker::findAnagrams (string soFar, lettersStruct input[], bool
print){
    vector<string>::iterator dictCur = dictBegin;
    if (dictCur==dictEnd) return 0;

    int count=0;
    while (dictCur!=dictEnd){
        //for each word X in
dictionary
        if (dictCur[0][0]=='/') dictCur++; //Cur==dictCur/* &&
Cur!=dictEnd*/) Cur++;
        if (compare(*dictCur, input)){
            //if input has that
letter in it, decrement and add to 'soFar'
            if (soFar=="") soFar+=*dictCur;
            else soFar += (" " + *dictCur); //add (append) X to the
end of soFar

            *dictCur = "/" + *dictCur;
            dictCur++;

            if (input[0].length<=0){
                /*if (difference == 0 letters left) {found an anagram
since no letters left*/
                count++;
                if (print)cout << soFar << endl;
                //return findAnagrams(soFar, inputArr, print, count);
            }
            else {
                dictCur++;
                count += findAnagrams(soFar, input, print);
            }

            soFar = subtractString(soFar, input);
            //REMOVE X FROM THE END OF 'soFar'
        }
        dictCur++;
        if (dictCur==dictEnd && input[0].length > 0){
            soFar = subtractString(soFar, input);
            dictCur=dictBegin;
        }
        if (soFar==" " && count>0){
            resetDict(dictCur);
            dictCur=dictBegin;
        }
    }
    //RECURSE TO THE NEXT LEVEL HERE.
    /*W O R K   I N   P R O G R E S S*/
    }
    return count;
}

int AnagramMaker::findAnagrams(string input, bool print){//initializes all
used variables and arrays
    if (input.empty()) return 0;
    int inputlen = input.length();
    for (int i=0; i<inputlen; i++){
        //for loop
        increments the count of each letter for the array.
        if (inputArr[(int)input[i]-'a'].count==0){
            //each letter
            represents a number in the array (eg. a=0 b=1 c=2 etc...) and

```

StopWatch.h

```
#pragma once
//This class comes from David Bolton,
//http://cplus.about.com/od/howtothingsi2/a/timing.htm
//C version is at http://cplus.about.com/od/howtothingsin1/a/timing.htm
//supposed to be accurate to microsecond
//possible dual core issues,
//but this code won't run on multiple cores simultaneously anyway
#include <windows.h>

typedef struct {
    LARGE_INTEGER start;
    LARGE_INTEGER stop;
} stopWatch;

class CStopWatch {
private:
    stopWatch timer;
    LARGE_INTEGER frequency;
    double LIToSecs( LARGE_INTEGER & L) ;
public:
    CStopWatch() ;
    void startTimer( ) ;
    void stopTimer( ) ;
    double getElapsedTime() ;
};
```

My Online Store

COINS 332

Applied Networking

Grade: 85/100

Language written: ASP.Net, VB.Net

Compiler: Microsoft Visual Studio 2008

How to run: Must be run from Visual Studio 2008+ unless IIS is enabled on local computer.
When ran, logging in is enabled after creating a new user.

Default.aspx

<!--
Markowitz Frost

Web Store

Created November 16, 2009

Last Modified: December 7, 2009

COIN Course: COINS 332

Grade Received: 87/100

Comments regarding design:

ASP.NET web store that requires users to log in, stores users in the ASP database. Security was implemented into the web pages that require a logged in user to be accessed. Option to create a new user (register) from the storefront was also added along with a recover password ability. For the store, when purchasing, after someone's made their selection via the checkboxes then they can press the checkout button which stores their order with their user ID for purchase history.

-->

```
<%@ Page Language="VB" AutoEventWireup="false" CodeFile="Default.aspx.vb"
Inherits="_Default" %>
```

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
```

```
<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
```

```
    <title>Untitled Page</title>
```

```
    <style type="text/css">
```

```
        .style2
```

```
        {
```

```
            font-size: xx-large;
```

```
        }
```

```
        .style3
```

```
        {
```

```
            text-decoration: underline;
```

```
            color: #0000FF;
```

```
        }
```

```
        .style5
```

```
        {
```

```
            font-family: "Blackadder ITC";
```

```
        }
```

```
    </style>
```

```
</head>
```

```
<body style="background-color: #000000">
```

```
    <form id="form1" runat="server">
```

```
        <div>
```

```
            <center><font color="red" size="18" style="font-size: large"><span
class="style2">Welcome to
```

```
                Planet Audio!</span><br />
```

```
                <span class="style5">Where the best come to buy the
best</span><br />
```

Default.aspx

```
<asp:Login ID="Login1" runat="server" BackColor="#F7F6F3"
BorderColor="#E6E2D8"
    BorderPadding="4" BorderStyle="Solid" BorderWidth="1px" Font-
Names="Verdana"
    Font-Size="0.8em" ForeColor="#333333" Width="344px"
    CreateUserUrl="~/NewUser.aspx"
    DestinationPageUrl="~/Login_successful.aspx">
    <TextBoxStyle Font-Size="0.8em" />
    <LoginButtonStyle BackColor="#FFFBFF" BorderColor="#CCCCCC"
BorderStyle="Solid"
    BorderWidth="1px" Font-Names="Verdana" Font-Size="0.8em"
ForeColor="#284775" />
    <InstructionTextStyle Font-Italic="True" ForeColor="Black" />
    <TitleTextStyle BackColor="#5D7B9D" Font-Bold="True" Font-
Size="0.9em"
    ForeColor="White" />
</asp:Login>
<br />
Or are you a <a href="NewUser.aspx"><span class="style3">new
user</span></a>?<br />
</font></center></div>
</form>
</body>
</html>
```

Default.aspx.vb

```
Partial Class _Default
    Inherits System.Web.UI.Page

    Protected Sub Login1_Authenticate(ByVal sender As Object, ByVal e As
System.Web.UI.WebControls.AuthenticateEventArgs) Handles Login1.Authenticate
        'FormsAuthentication.Authenticate(Login1.UserName.ToString(),
Login1.Password)
        'Response.Write(Login1.Password)
        Session("UserName") = Login1.UserName.ToString()
    End Sub
End Class
```

Login_Successful.aspx

```
<%@ Page Language="VB" AutoEventWireup="false"
CodeFile="Login_successful.aspx.vb" Inherits="Login_successful" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
    <title>Untitled Page</title>
    <style type="text/css">
        .style1
        {
            color: #0000FF;
            text-decoration: underline;
        }
        .style2
        {
            color: #FF0000;
        }
    </style>
</head>
<body style="text-align: center; background-color: #000000;">
    <form id="form1" runat="server">
        <div>

            <span class="style2">Congratulations you have successfully logged
in</span><br>
                class="style2" />
            <br class="style2" />
            <span class="style2">Please wait while we process you.</span><br>
                class="style2" />
            <br class="style2" />
            <span class="style2">.....</span><br class="style2" />
            <br class="style2" />
            <span class="style2">If the page does not redirect in the next 5 to
10 seconds please click the
following link.</span><br class="style2" />
            <asp:ScriptManager ID="ScriptManager1" runat="server">
</asp:ScriptManager>
            <br />
            <span class="style1">
<a href="Store\StoreFront.aspx">Click here to procede</a></span><br>
/>

            <asp:Timer ID="Timer1" runat="server" Interval="3000">
</asp:Timer>
        </div>
    </form>
</body>
</html>
```

Login_successful.aspx.vb

```
Partial Class Login_successful
    Inherits System.Web.UI.Page

    Protected Sub Timer1_Tick(ByVal sender As Object, ByVal e As
System.EventArgs) Handles Timer1.Tick
        Response.Redirect("Store/StoreFront.aspx")
    End Sub

    Protected Sub Page_Load(ByVal sender As Object, ByVal e As
System.EventArgs) Handles Me.Load
        Timer1.Enabled() = True
    End Sub
End Class
```



Login_failed.aspx.vb

```
Partial Class Login_failed  
    Inherits System.Web.UI.Page
```

```
End Class
```

NewUser.aspx

```
<%@ Page Language="VB" AutoEventWireup="false" CodeFile="NewUser.aspx.vb"
Inherits="NewUser" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
    <title>Untitled Page</title>
    <style type="text/css">
        .style1
        {
            color: #FF0000;
        }
        .style2
        {
            color: #FF0000;
            font-size: 46pt;
        }
    </style>
</head>
<body style="background-color: #000000">
    <form id="form1" runat="server">
        <div style="text-align: center">

            <span class="style2">Welcome New User!</span><br class="style1" />
            <asp:SqlDataSource ID="SqlDataSource1" runat="server"
                ConnectionString="<%%$ ConnectionStrings:ConnectionString %>"
                DeleteCommand="DELETE FROM [UsersTable] WHERE [UserID] = @UserID"
                InsertCommand="INSERT INTO [UsersTable] ([UserName], [Email])
VALUES (@UserName, @Email)"
                SelectCommand="SELECT * FROM [UsersTable]"
                UpdateCommand="UPDATE [UsersTable] SET [UserName] = @UserName,
[Email] = @Email WHERE [UserID] = @UserID">
                <DeleteParameters>
                    <asp:Parameter Name="UserID" Type="Int32" />
                </DeleteParameters>
                <UpdateParameters>
                    <asp:Parameter Name="UserName" Type="String" />
                    <asp:Parameter Name="Email" Type="String" />
                    <asp:Parameter Name="UserID" Type="Int32" />
                </UpdateParameters>
                <InsertParameters>
                    <asp:Parameter Name="UserName" Type="String" />
                    <asp:Parameter Name="Email" Type="String" />
                </InsertParameters>
            </asp:SqlDataSource>
            <br />
            <center>
                <asp:CreateUserWizard ID="CreateUserWizard1" runat="server"
                    BackColor="#E6F3FB"
                    BorderColor="#B5C7DE" BorderStyle="Solid" BorderWidth="1px"
                    ContinueDestinationPageUrl="~/Login_successful.aspx" Font-
Names="Verdana"
                    Font-Size="0.8em" Height="327px" style="font-size: medium"
                    Width="289px">
```

NewUser.aspx

```

        <SideBarStyle BackColor="#507CD1" Font-Size="0.9em"
VerticalAlign="Top" />
        <SideBarButtonStyle BackColor="#507CD1" Font-Names="Verdana"
        ForeColor="White" />
        <ContinueButtonStyle BackColor="White" BorderColor="#507CD1"
        BorderStyle="Solid" BorderWidth="1px" Font-Names="Verdana"
        ForeColor="#284E98" />
        <NavigationButtonStyle BackColor="White" BorderColor="#507CD1"
        BorderStyle="Solid" BorderWidth="1px" Font-Names="Verdana"
        ForeColor="#284E98" />
        <HeaderStyle BackColor="#284E98" BorderColor="#EFF3FB"
BorderStyle="Solid"
        BorderWidth="2px" Font-Bold="True" Font-Size="0.9em"
ForeColor="White"
        HorizontalAlign="Center" />
        <CreateUserButtonStyle BackColor="White" BorderColor="#507CD1"
        BorderStyle="Solid" BorderWidth="1px" Font-Names="Verdana"
        ForeColor="#284E98" />
        <TitleTextStyle BackColor="#507CD1" Font-Bold="True"
ForeColor="White" />
        <StepStyle Font-Size="0.8em" />
        <WizardSteps>
            <asp:CreateUserWizardStep runat="server">
                <ContentTemplate>
                    <table border="0" style="font-family:Verdana;font-
size:100%;">
                        <tr>
                            <td align="center" colspan="2"
                                style="color:White;background-
color:#507CD1;font-weight:bold;">
                                Sign Up for Your New Account</td>
                        </tr>
                        <tr>
                            <td align="right">
                                <asp:Label ID="UserNameLabel"
runat="server" AssociatedControlID="UserName"
                                style="font-size: small">User
Name:</asp:Label>
                            </td>
                            <td>
                                <asp:TextBox ID="UserName"
runat="server"></asp:TextBox>
                                <asp:RequiredFieldValidator
ID="UserNameRequired" runat="server"
                                ControlToValidate="UserName"
                                ErrorMessage="User Name is required."
                                ToolTip="User Name is required."
                                ValidationGroup="CreateUserWizard1">*</asp:RequiredFieldValidator>
                            </td>
                        </tr>
                        <tr>
                            <td align="right">
                                <asp:Label ID="PasswordLabel"
runat="server" AssociatedControlID="Password"
                                style="font-size:
small">Password:</asp:Label>
                            </td>

```

NewUser.aspx

```

        <td>
            <asp:TextBox ID="Password" runat="server"
TextMode="Password"></asp:TextBox>
            <asp:RequiredFieldValidator
ID="PasswordRequired" runat="server"
                ControlToValidate="Password"
ErrorMessage="Password is required."
                ToolTip="Password is required."
ValidationGroup="CreateUserWizard1">*</asp:RequiredFieldValidator>
        </td>
    </tr>
    <tr>
        <td align="right">
            <asp:Label ID="ConfirmPasswordLabel"
runat="server"
                AssociatedControlID="ConfirmPassword"
style="font-size: small">Confirm
                Password:</asp:Label>
        </td>
        <td>
            <asp:TextBox ID="ConfirmPassword"
runat="server" TextMode="Password"></asp:TextBox>
            <asp:RequiredFieldValidator
ID="ConfirmPasswordRequired" runat="server"
                ControlToValidate="ConfirmPassword"
ErrorMessage="Confirm Password is
required."
                ToolTip="Confirm Password is
required." ValidationGroup="CreateUserWizard1">*</asp:RequiredFieldValidator>
        </td>
    </tr>
    <tr>
        <td align="right">
            <asp:Label ID="EmailLabel" runat="server"
AssociatedControlID="Email"
                style="font-size: small">E-
mail:</asp:Label>
        </td>
        <td>
            <asp:TextBox ID="Email"
runat="server"></asp:TextBox>
            <asp:RequiredFieldValidator
ID="EmailRequired" runat="server"
                ControlToValidate="Email"
ErrorMessage="E-mail is required."
                ToolTip="E-mail is required."
ValidationGroup="CreateUserWizard1">*</asp:RequiredFieldValidator>
        </td>
    </tr>
    <tr>
        <td align="right">
            <asp:Label ID="QuestionLabel"
runat="server" AssociatedControlID="Question"
                style="font-size: small">Security
                Question:</asp:Label>
        </td>
        <td>

```

NewUser.aspx

```

runat="server"></asp:TextBox>
        <asp:RequiredFieldValidator
ID="QuestionRequired" runat="server"
        ControlToValidate="Question"
ErrorMessage="Security question is required."
        ToolTip="Security question is
required." ValidationGroup="CreateUserWizard1">*</asp:RequiredFieldValidator>
    </td>
</tr>
<tr>
    <td align="right">
        <asp:Label ID="AnswerLabel"
runat="server" AssociatedControlID="Answer"
        style="font-size: small">Security
Answer:</asp:Label>
    </td>
    <td>
        <asp:TextBox ID="Answer" runat="server"
Width="128px"></asp:TextBox>
        <asp:RequiredFieldValidator
ID="AnswerRequired" runat="server"
        ControlToValidate="Answer"
ErrorMessage="Security answer is required."
        ToolTip="Security answer is
required." ValidationGroup="CreateUserWizard1">*</asp:RequiredFieldValidator>
    </td>
</tr>
<tr>
    <td align="center" colspan="2">
        <asp:CompareValidator
ID="PasswordCompare" runat="server"
        ControlToCompare="Password"
ControlToValidate="ConfirmPassword"
        Display="Dynamic"
ErrorMessage="The Password and
Confirmation Password must match."
        style="font-size: small"
ValidationGroup="CreateUserWizard1"></asp:CompareValidator>
    </td>
</tr>
<tr>
    <td align="center" colspan="2">
        style="color:Red; font-size: small;">
        <asp:Literal ID="ErrorMessage"
runat="server" EnableViewState="False"></asp:Literal>
    </td>
</tr>
</table>
</ContentTemplate>
</asp:CreateUserWizardStep>
<asp:CompleteWizardStep runat="server" >
    <ContentTemplate>
        <table border="0"
            style="font-family:Verdana;font-
size:100%;height:327px;width:289px;">
            <tr>

```

NewUser.aspx

```

        <td align="center" colspan="2"
color:#507CD1;font-weight:bold;">
            Complete</td>
        </tr>
        <tr>
            <td>
                Your account has been successfully
            </td>
        </tr>
        <tr>
            <td align="right" colspan="2">
                <asp:Button ID="ContinueButton"
runat="server" BackColor="White"
                    BorderColor="#507CD1"
                    BorderStyle="Solid" BorderWidth="1px"
                    CausesValidation="False"
                    CommandName="Continue" Font-Names="Verdana"
                    ForeColor="#284E98" Text="Continue"
                    ValidationGroup="CreateUserWizard1" />
            </td>
        </tr>
    </table>
</ContentTemplate>
</asp:CompleteWizardStep>
</WizardSteps>
</asp:CreateUserWizard></center>

</div>
</form>
</body>
</html>
```

NewUser.aspx.vb

```
Imports System
Imports System.Data
Imports System.Data.Sql
Imports System.Data.SqlClient

Partial Class NewUser
    Inherits System.Web.UI.Page

    Protected Sub CreateUserWizard1_CreatedUser(ByVal sender As Object, ByVal
e As System.EventArgs) Handles CreateUserWizard1.CreatedUser
        Dim UserName As String
        Dim Email As String

        UserName = CreateUserWizard1.UserName.ToString()
        Email = CreateUserWizard1.Email

        SqlDataSource1.InsertParameters("UserName").DefaultValue = UserName
        Session("UserName") = UserName
        SqlDataSource1.InsertParameters("Email").DefaultValue = Email

    End Sub
End Class
```

Logout.aspx

```
<%@ Page Language="VB" AutoEventWireup="false" CodeFile="Logout.aspx.vb"
Inherits="Logout" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
    <title>Untitled Page</title>
    <style type="text/css">
        .style1
        {
            color: #FF0000;
        }
    </style>
</head>
<body style="background-color: #000000">
    <form id="form1" runat="server">
        <div style="text-align: center">

            <span class="style1">Please Wait </span>
            <br class="style1" />
            <asp:ScriptManager ID="ScriptManager1" runat="server">
            </asp:ScriptManager>
            <asp:Timer ID="Timer1" runat="server" EnableViewState="False"
Interval="800">
            </asp:Timer>
            <br class="style1" />
            <span class="style1">.....</span><br class="style1" />
            <br class="style1" />
            <span class="style1">while we log you out.</span></div>
        </form>
    </body>
</html>
```

LogOut.aspx.vb

```
Partial Class LogOut
    Inherits System.Web.UI.Page

    Protected Sub Timer1_Tick(ByVal sender As Object, ByVal e As
System.EventArgs) Handles Timer1.Tick
        Response.Redirect("default.aspx")
    End Sub

    Protected Sub Page_Load(ByVal sender As Object, ByVal e As
System.EventArgs) Handles Me.Load
        Timer1.Enabled() = True
        FormsAuthentication.SignOut()
    End Sub
End Class
```

Store/Invoice.aspx

```

<%@ Page Language="VB" AutoEventWireup="false" CodeFile="Invoice.aspx.vb"
Inherits="Store_Invoice" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
    <title>Untitled Page</title>
    <style type="text/css">
        .style1
        {
            font-size: 46pt;
        }
        .style2
        {
            font-size: medium;
        }
        .style3
        {
            color: #FF0000;
        }
    </style>
</head>
<body style="background-color: #000000">
    <form id="form1" runat="server">
        <div class="style1" style="text-align: center">

            <span class="style3">INVOICE</span><br class="style3" />
            <asp:SqlDataSource ID="SqlDataSource1" runat="server"
                ConnectionString="<*$ ConnectionStrings:ConnectionString %>"

                SelectCommand="SELECT [ID], [UserName], [Item1], [Item2],
[Item3], [Item4], [Item5], [Item6], [Price] FROM [Cart] WHERE ([UserName] =
@UserName)">
                <SelectParameters>
                    <asp:SessionParameter Name="UserName" SessionField="UserName"
Type="String" />
                </SelectParameters>
            </asp:SqlDataSource>
            <br />
            <center></center><asp:GridView ID="GridView1" runat="server"
                AutoGenerateColumns="False" CellPadding="4" DataKeyNames="ID"
                DataSourceID="SqlDataSource1" ForeColor="#333333"
GridLines="None"
                style="font-size: medium">
                <RowStyle BackColor="#EFF3FB" />
                <Columns>
                    <asp:BoundField DataField="ID" HeaderText="ID"
                        InsertVisible="False" ReadOnly="True" SortExpression="ID"
                    />
                    <asp:BoundField DataField="UserName" HeaderText="UserName"
                        SortExpression="UserName" />
                    <asp:BoundField DataField="Item1" HeaderText="Item1"
SortExpression="Item1" />
                    <asp:BoundField DataField="Item2" HeaderText="Item2"
SortExpression="Item2" />

```

Store/Invoice.aspx

```

        <asp:BoundField DataField="Item3" HeaderText="Item3"
SortExpression="Item3" />
        <asp:BoundField DataField="Item4" HeaderText="Item4"
SortExpression="Item4" />
        <asp:BoundField DataField="Item5" HeaderText="Item5"
SortExpression="Item5" />
        <asp:BoundField DataField="Item6" HeaderText="Item6"
SortExpression="Item6" />
        <asp:BoundField DataField="Price" HeaderText="Price"
SortExpression="Price" />
    </Columns>
    <FooterStyle BackColor="#507CD1" Font-Bold="True"
ForeColor="White" />
    <PagerStyle BackColor="#2461BF" ForeColor="White"
HorizontalAlign="Center" />
    <SelectedRowStyle BackColor="#D1DDF1" Font-Bold="True"
ForeColor="#333333" />
    <HeaderStyle BackColor="#507CD1" Font-Bold="True"
ForeColor="White" />
    <EditRowStyle BackColor="#2461BF" />
    <AlternatingRowStyle BackColor="White" />
</asp:GridView></center>

</div>
</form>
</body>
</html>
```

StoreFront.aspx

```
<%@ Page Language="VB" AutoEventWireup="false" CodeFile="StoreFront.aspx.vb"
Inherits="Store_StoreFront" %>
```

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
```

```
<html xmlns="http://www.w3.org/1999/xhtml">
```

```
<head runat="server">
```

```
  <title>Untitled Page</title>
```

```
  <style type="text/css">
```

```
    .style1
```

```
    {
```

```
      font-size: medium;
```

```
    }
```

```
    .style3
```

```
    {
```

```
      font-size: 24pt;
```

```
    }
```

```
    .style4
```

```
    {
```

```
      background-color: #0033CC;
```

```
    }
```

```
    .style5
```

```
    {
```

```
      font-size: medium;
```

```
      background-color: #0033CC;
```

```
    }
```

```
    .style6
```

```
    {
```

```
      background-color: #0066FF;
```

```
    }
```

```
    .style7
```

```
    {
```

```
      font-size: medium;
```

```
      background-color: #0066FF;
```

```
    }
```

```
    .style8
```

```
    {
```

```
      background-color: #6699FF;
```

```
    }
```

```
    .style9
```

```
    {
```

```
      font-size: medium;
```

```
      background-color: #6699FF;
```

```
    }
```

```
    .style10
```

```
    {
```

```
      background-color: #003399;
```

```
    }
```

```
    .style11
```

```
    {
```

```
      background-color: #CCCCCC;
```

```
    }
```

```
    .style12
```

```
    {
```

```
      font-size: medium;
```

```
      background-color: #CCCCCC;
```

StoreFront.aspx

```

    }
    .style13
    {
        background-color: #999999;
    }
    .style14
    {
        font-size: medium;
        background-color: #999999;
    }
    .style15
    {
        background-color: #666666;
    }
    .style16
    {
        font-size: medium;
        background-color: #666666;
    }
    .style17
    {
        color: #FF0000;
    }
    .style18
    {
        color: #FF0000;
        font-size: 36pt;
    }
</style>
</head>
<body style="background-color: #000000">
    <form id="form1" runat="server">
        <div class="style1" style="text-align: center">

            <span class="style18">The Store That Dreams Are Made Of....</span><br
            class="style17" />
            <asp:SqlDataSource ID="SqlDataSource1" runat="server"
                ConnectionString="<%= $ ConnectionStrings:ConnectionString %>"
                SelectCommand="SELECT * FROM [Item List]">
            </asp:SqlDataSource>
            <asp:SqlDataSource ID="SqlDataSourceCart" runat="server"
                ConnectionString="<%= $ ConnectionStrings:ConnectionString %>"
                DeleteCommand="DELETE FROM [Cart] WHERE [ID] = @ID"
                InsertCommand="INSERT INTO [Cart] ([UserName], [Item1], [Item3],
                [Item2], [Item6], [Item4], [Item5], [Price]) VALUES (@UserName, @Item1,
                @Item3, @Item2, @Item6, @Item4, @Item5, @Price)"
                SelectCommand="SELECT [ID], [UserName], [Item1], [Item3],
                [Item2], [Item6], [Item4], [Item5], [Price] FROM [Cart]"

                UpdateCommand="UPDATE [Cart] SET [UserName] = @UserName, [Item1]
                = @Item1, [Item3] = @Item3, [Item2] = @Item2, [Item6] = @Item6, [Item4] =
                @Item4, [Item5] = @Item5, [Price] = @Price WHERE [ID] = @ID">
                <DeleteParameters>
                    <asp:Parameter Name="ID" Type="Int32" />
                </DeleteParameters>
                <UpdateParameters>
                    <asp:Parameter Name="UserName" Type="String" />

```

StoreFront.aspx

```

        <asp:Parameter Name="Item1" Type="String" />
        <asp:Parameter Name="Item3" Type="String" />
        <asp:Parameter Name="Item2" Type="String" />
        <asp:Parameter Name="Item6" Type="String" />
        <asp:Parameter Name="Item4" Type="String" />
        <asp:Parameter Name="Item5" Type="String" />
        <asp:Parameter Name="Price" Type="String" />
        <asp:Parameter Name="ID" Type="Int32" />
    </UpdateParameters>
    <InsertParameters>
        <asp:Parameter Name="UserName" Type="String" />
        <asp:Parameter Name="Item1" Type="String" />
        <asp:Parameter Name="Item3" Type="String" />
        <asp:Parameter Name="Item2" Type="String" />
        <asp:Parameter Name="Item6" Type="String" />
        <asp:Parameter Name="Item4" Type="String" />
        <asp:Parameter Name="Item5" Type="String" />
        <asp:Parameter Name="Price" Type="String" />
    </InsertParameters>
</asp:SqlDataSource>
<a href="../LogOut.aspx" style="text-align: right">Log Out</a>
<br />

<br />

<br />
<table style="width:100%;">
    <tr>
        <span class ="style3">
            <td class="style10">
                Snapshot</td>
            <td class="style10">
                Name</td>

            <td class="style10">
                ModelNumber</td>
            <td class="style10">
                Description</td>
            <td class="style10">
                Type</td>
            <td class="style10">
                Price</td>
            <td class="style10">
                Purchase</span></td>
        </tr>
        <tr>
            <td class="style11">
                </td>
            <td class="style12">
                Eclipse AVN7000</td>
            <td class="style11">
                ECE AVN7000</td>
            <td class="style11">
                6.5" flip-screen touch screen media player</td>
            <td class="style11">
                CD Player</td>
            <td class="style11">

```

StoreFront.aspx

```

        $599.99</td>
<td class="style11">
    </span>
    <asp:CheckBox ID="CheckBox1" runat="server" />
</td>
</tr>
<tr>
    <td class="style4">
        </td>
    <td class="style5">
        Pioneer Premier    <td class="style4">
        PIO DEHP980BT</td>
    <td class="style4">
        CD/MP3 player with Aux input</td>
    <td class="style4">
        CD Player</td>
    <td class="style4">
        $269.99</td>
    <td class="style4">
        </span>
        <asp:CheckBox ID="CheckBox2" runat="server" />
    </td>
</tr>
<tr>
    <td class="style13">
        </td>
    <td class="style14">
        Kicker 6.5" Component speakers    <td
class="style13">
        KKR DS650.2</td>
    <td class="style13">
        6-3/4" 2-Way Component System</td>
    <td class="style13">
        Speakers</td>
    <td class="style13">
        $79.99</td>
    <td class="style13">
        </span>
        <asp:CheckBox ID="CheckBox3" runat="server" />
    </td>
</tr>
<tr>
    <td class="style6">
        </td>
    <td class="style7">
        Alpine 6.5" Component System    <td class="style6">
        APN SPX17REF</td>
    <td class="style6">
        6.5" Type X Series Component System</td>
    <td class="style6">
        Speakers</td>
    <td class="style6">
        $179.99</td>
    <td class="style6">
        </span>
        <asp:CheckBox ID="CheckBox4" runat="server" />
    </td>
</tr>

```

```

</tr>
<tr>
  <td class="style15">
    </td>
  <td class="style16">
    MTX Thunder Elite    <td class="style15">
      MTX 4001D</td>
  <td class="style15">
    7500W Max Class D Monoblock Amplifier</td>
  <td class="style15">
    Amplifier</td>
  <td class="style15">
    $1999.99</td>
  <td class="style15">
    </span>
    <asp:CheckBox ID="CheckBox5" runat="server" />
  </td>
</tr>
<tr>
  <td class="style8">
    </td>
  <td class="style9">
    JL Audio Slash v2 1000/1v2e8"
  <td class="style8">
    JLA 1000/1v2</td>
  <td class="style8">
    1000W RMS Slash Version 2 Series Amplifier</td>
  <td class="style8">
    Amplifier</td>
  <td class="style8">
    $659.99</td>
  <td class="style8">
    </span>
    <asp:CheckBox ID="CheckBox6" runat="server" />
  </td>
</tr>
</table>

<br />
<asp:Button ID="Button1" runat="server" Text="Add To Cart"
Width="162px" />

</div>
</form>
</body>
</html>

```

StoreFront.aspx.vb

```
Imports System.Data
Imports System.Data.Sql
Imports System.Data.SqlClient

Partial Class Store_StoreFront
    Inherits System.Web.UI.Page

    Protected Sub Button1_Click(ByVal sender As Object, ByVal e As
System.EventArgs) Handles Button1.Click
        Dim price As Double
        'Dim i As Integer
        'Dim test As String
        'Dim reader As SqlDataReader
        price = 0.0
        If CheckBox1.Checked Then
            SqlDataSourceCart.InsertParameters("Item1").DefaultValue =
"Eclipse AVN7000"
            'For i = 0 To 6
            '    test = "item" + CStr(i)
            '    If SqlDataSource1.SelectParameters(test).DefaultValue =
"NULL" Then
                '        SqlDataSource1.InsertParameters(test).DefaultValue =
"Eclipse AVN7000"
            '    End If
            'Next
            price = price + 599.99
        End If
        If CheckBox2.Checked Then
            SqlDataSourceCart.InsertParameters("Item2").DefaultValue =
"Pioneer Premier"
            'For i = 0 To 6
            '    test = "item" + i
            '    If SqlDataSource1.SelectParameters(test).DefaultValue =
"NULL" Then
                '        SqlDataSource1.InsertParameters(test).DefaultValue =
"Pioneer Premier"
            '    End If
            'Next
            price = price + 269.99
        End If
        If CheckBox3.Checked Then
            SqlDataSourceCart.InsertParameters("Item3").DefaultValue =
"Kicker 6.5inch Component speakers"
            'For i = 0 To 6
            '    test = "item" + i
            '    If SqlDataSource1.SelectParameters(test).DefaultValue =
"NULL" Then
                '        SqlDataSource1.InsertParameters(test).DefaultValue =
"Kicker 6.5" Component speakers"
            '    End If
            'Next
            price = price + 79.99
        End If
        If CheckBox4.Checked Then
            SqlDataSourceCart.InsertParameters("Item4").DefaultValue =
"Alpine 6.5inch Component System"
            'For i = 0 To 6
```

StoreFront.aspx.vb

```
'      test = "item" + i
'      If SqlDataSource1.SelectParameters(test).DefaultValue =
"NULL" Then
'          SqlDataSource1.InsertParameters(test).DefaultValue =
"Alpine 6.5" Component System"
'      End If
'Next
price = price + 179.99
End If
If CheckBox5.Checked Then
    SqlDataSourceCart.InsertParameters("Item5").DefaultValue = "MTX
Thunder Elite"
'For i = 0 To 6
'    test = "item" + i
'    If SqlDataSource1.SelectParameters(test).DefaultValue =
"NULL" Then
'        SqlDataSource1.InsertParameters(test).DefaultValue =
"MTX Thunder Elite"
'    End If
'Next
price = price + 1999.99
End If
If CheckBox6.Checked Then
    SqlDataSourceCart.InsertParameters("Item6").DefaultValue = "JL
Audio Slash v2 1000/1v2"
'For i = 0 To 6
'    test = "item" + i
'    If SqlDataSource1.SelectParameters(test).DefaultValue =
"NULL" Then
'        SqlDataSource1.InsertParameters(test).DefaultValue = "JL
Audio Slash v2 1000/1v2"
'    End If
'Next
price = price + 659.99
End If
SqlDataSourceCart.InsertParameters("Price").DefaultValue = price
SqlDataSourceCart.InsertParameters("UserName").DefaultValue =
Session("UserName")
SqlDataSourceCart.Insert()
Response.Redirect("Invoice.aspx")
End Sub
End Class
```

1-bit counter

COINS 333

Applied Systems

Grade: 13/17

Language written: Assembly/MIPS

Compiler: QTSpim or PCSpim

How to run: When ran from QTSpim or PCSpim, the dialogue box asks for you to enter a number then, do so then press 'Enter'. Afterwards the code generates the output:

Number of 1 bits: #

Position of lowest 1 bit: #

Position of highest 1 bit: #

hw3-1.s

```
# UNTITLED PROGRAM
#Markowitz Frost
```

```
#Count the 1 bits
```

```
#Created December 9, 2009
```

```
#Last Modified: December 14, 2009
```

```
#COIN Course: COINS 333
```

```
#Grade Received: 13/17
```

```
#An assembly program that reads a number from the keyboard. It then
#prints out the number of 1 bits in the number. It then prints out
#the lowest and highest positions of a 1 bit where 0 represents the
#lowest order (furthest right) bit and 31 represents the highest.
#It prints 1 if there are no 1 bits in the number.
```

```
        .data          # Data declaration section
ENT:     .asciiz "Enter a number: "
NUMBER:  .asciiz "Number of 1 bits: "
POSL:    .asciiz "\nPosition of lowest 1 bit: "
POSH:    .asciiz "\nPosition of highest 1 bit: "
        .text

main:    # Start of code section
        li            $v0, 4
        la            $a0, ENT
        syscall
        li            $v0, 5
        syscall
        move          $s0, $v0
        li            $t0, 2
        li            $s2, -1
        li            $s3, -1

# $s1=#OfBits $s2=Low $s3=High
LOOP:    div          $s0, $t0
        mflo          $s0      #save quotient
        mfhi          $t1      #save remainder for checking
        blez          $s0, AFTERLOOP
        blez          $t1, LOOP
#If remainder is 0 keep dividing
MOVEHI:  addi          $s3, 1 #highest from remainder 1
        addi          $s1, 1

LOOP2:   div          $s0, $t0
        mflo          $s0      #save quotient
        mfhi          $t1      #save remainder for checking
        addi          $s3, 1    #moved spots, highest is now 1 spot further
        beq           $t1, 1, MOVELO
        blez          $s0, AFTERLOOP
        blez          $t1, LOOP2
MOVELO:  mfhi          $s2      #lowest from remainder 1
        addi          $s1, 1
        li            $t1, 0
```

```
        j        LOOP2

AFTERLOOP:
        li        $v0, 4
        la        $a0, NUMBER
        syscall
        li        $v0, 1
        move      $a0, $s1
        syscall

        li        $v0, 4
        la        $a0, POSL
        syscall
        li        $v0, 1
        move      $a0, $s2
        syscall

        li        $v0, 4
        la        $a0, POSH
        syscall
        li        $v0, 1
        move      $a0, $s3
        syscall

        li        $v0, 10
        syscall

# END OF PROGRAM
```

Ethics Paper

COINS 409

Artificial Intelligence

Grade: 80/100

Artificial Intelligence

COINS 409

February 9, 2011

Mark Frost

The next step in the evolution of the society that humans live in will not be built solely by humans; it will be built with the assistance of Artificial Intelligence (A.I.). The general background we normally understand when someone presents the idea of AI implementation into a robot or a computer program is that the goal is to simulate the human thought process. However, the question is why would we want to build them to be 'human' like? If we succeed at developing an AI that is 'human' like, then how would they be considered among our society, as a human-like robot, or as a tool? There must be guidelines for AI to follow and continue to develop over the lifetime of its 'existence' for humans to accept the idea of implementation; this would be the human equivalent of our moral system. Some authors believe that A.I. is not an accurate term to label the idea of what it is. David Poole and his colleagues believe that the term Artificial intelligence can be confusing and should be labeled as computational intelligence.

The reason for deciding to use the term 'computational intelligence' is, "since we claimed that the central scientific goal is to understand both natural and artificial (or synthetic) systems, we prefer the name 'computational intelligence.' It also has the advantage of making the computational hypothesis explicit in the name." (Poole pg 1-2). The idea suggested is plausible because, though we have made strides in the current era of A.I., we have not made it to the point of 'creating' the human intelligence, or thought process; which is what the term Artificial Intelligence infers. An idea also presented is that there are classes of intelligence and the class that is more intelligent than humans is the class of 'organizations'. This idea was brought about by the disconnect between the effective intelligence of a single human versus the effective intelligence of a collective of humans performing complex tasks. Someone who has another take on the AI idea is John McCarthy, a major contributor to the field, he sees Artificial Intelligence as, "the science and engineering of making intelligent machines." (McCarthy). The term of

'intelligent machines' covers a broad spectrum of devices like computer programs that records habits and remembers the things you want to do can seem intelligent, but so can a robot who you can sit and have a physical conversation with. McCarthy believes, "Intelligence involves mechanisms, and AI research has discovered how to make computers carry out some of the and not others." (McCarthy). The key phrase 'involves mechanisms' coincides with Poole's view of being a collective of intelligent agents that should be used to create a more accurate and useful representation of AI. The goal that is trying to be attained seems to be similar to a neural network, trying to emulate the way our brains function and think so future AI can be more human-like.

Artificial neural networks are usually built around programs that emulate the properties and actions of biological neurons. Creation of these networks is a step in the right direction if we want the AI we interact with to be similar to us. Our goal to make AI human-like is going to be a long road; the means to meet that end goal is not clearly discerned or stated in publications on AI development, especially if you consider the idea that AI development has subfields that do not communicate with each other. Pamela McCorduck suggests that the subfields are divided among five major subfields, which are, "vision, natural language, decision theory, genetic algorithms, [and] robotics." (McCorduck). There is no general term or idea to classify AI as a whole; designers and programmers are taking many different approaches to attempt to bring the reality of useful AI to the people. Arguably, it is also up to the designers and programmers to make sure the AI are not going to go out of their programming boundaries to make sure that they can be accepted in society. If a rogue AI were to act on its own, the results could become catastrophic or be left at a minimum. The wide range in possibility of negative events happening is enough to

scare anyone off, besides John Connor, and leave the social stigma of AI being detrimental to humans behind.

The most basic moral responsibility of AI should be similar to that of a conscientious human that looks out for their fellow man. This base will be the foundation that they build their ethical or moral code from, it can never be avoided or erased. This means that careful thought and consideration for what that base should be will need to be taken into account to make sure there will be no paradoxes or conflicts between morals. The obvious first set of morals AI should have is Isaac Asimov's "Three Laws of Robotics". These laws imply three major points: It should not harm humans and along with protecting humans, it must obey orders from a human unless it conflicts with first rule, and it must protect its own existence if it does not conflict with first or second laws. Included in this base should be a rule for if a human asks for help within the AI's ability then it should comply with the condition that any requests of the owner of the AI is the first priority and does not conflict with the previous laws. As a final law for the base, an AI must know that it is an AI and must know its class in society. This is implied by the fifth law of robotics introduced by Nikola Kesarovski that states, "A robot must know it is a robot." (Kesarovski). The reason Kesarovski introduced the fifth law is because in his story the plot is about a robot that violated the first law and committed a murder because it did not know it was a robot. From that base it can develop its own set of principles based on its environment and events it experiences. Though these are relatively general safeguards, they present enough limitations for the AI to interact enough with its environment to determine what morals it should build upon or improve. In the end, it is our choice if we want or need the AI collective to benefit humanity externally by being just programs to help us through our daily lives or would we want a new

society of physical beings that were built by us. Either way, the reaction of the people is a coin flip, if implemented properly we could see the dawn of a new age.

Works Cited

J. J. Hopefield. "Neural networks and physical systems with emergent collective computational abilities." Proc. NatL Acad. Sci. USA Vol. 79, pp. 2554-2558, April 1982.

Kesarovski, Nikola. "The Fifth Law of Robotics." 1983.

McCarthy, John. "Basic Questions." Formal Reasoning Group. 12 Nov. 2007. Web. 12 Feb.

2011. <<http://www-formal.stanford.edu/jmc/whatisai/node1.html>>.

McCorduck, Pamela. Machines Who Think: a Personal Inquiry into the History and Prospects of Artificial Intelligence. Natick, MA: A.K. Peters, 2004. Print.

Poole, David L., Alan K. Mackworth, and Randy Goebel. Computational Intelligence: a Logical Approach. New York: Oxford UP, 1998. Print.

Skillings, Jonathan. "Getting Machines to Think like Us - CNET News." Technology News - CNET News. 3 July 2006. Web. <http://news.cnet.com/Getting-machines-to-think-like-us/2008-11394_3-6090207.html>.

Ethics Paper

COINS 315

Data Structures

Grade: 85/100

Mark Frost
Ethics Paper
Prof. Underwood
1/28/10

Discrimination is an issue that cannot be resolved soon, mainly because of the many forms that it shows up. In particular, the discrimination in the acceptance into universities has many factors, some that would not allow many people acceptance into an institute for reasons outside of their control. Now these discriminatory factors the universities use for applicants do vary by school but there is always a selection of factors that schools will most always use when weeding out the applicants that do not seem suitable for the institution.

One of the main things that the university will always look at is the personal statement that you write on their application. Usually this statement is around three to four sentences long that can explain the type of person you are or your hobbies and why you want to attend the institution. Admission counselors consider this when deciding if you should move on to the next step in the admission process. The GPA is also taken into consideration but in actuality, some of schools do not fully take consideration of just your GPA, some like Oxford look for well-rounded students with decent grades, extracurricular activities, and can show that they can excel in the field they plan to major in. However, these things can only go so far until references come into play; if a person reputable enough recommends you; admissions could easily change what they had originally prioritized you as before the recommendation. This blatant form of discrimination happens all the time to get people into schools they would have otherwise never been accepted to.

Next comes the experience portion of your application they look at to see if you have done anything with your life besides just make good grades or not. This portion includes work experience and extracurricular activities. In this area, admissions will look at what you have during your period of school prior to heading off to college, determining if you have held down a steady job or gotten involved with your school in some ways. Therefore, for the people who

thought that good grades alone, without any extracurricular activities, would be enough, they then would sometimes be put on a waiting list or be considered with the latter end of students because of their narrow background.

Some of the more prestigious schools like Oxford and Cambridge interview prospective students to find out what kind of person is reflected from the application. This is the perfect way for universities to weed you out from the rest of the competitive students trying to be accepted to the school. From this, it is also possible for them to determine if your social background has any positive or negative effects towards your possibility to being accepted. Factors such as your race, age, and locale of your hometown could be also taken into account at this point. But this could also get the institution into legal trouble if they were actively discriminating based on factors the student couldn't help.

The case of Regents of the University of California v. Bakke, in 1978, was one of where the University of California was actively reverse discriminating to fill their rigid quota of minorities. Bakke, a white applicant, was rejected two times from the university and there were minority students with significantly lower scores being accepted. The court ended up ruling that the university was admitting students while race was a legitimate factor in their acceptance.

The following case which occurred in 2003, Grutter v. Bollinger, the University of Michigan's law school was accused of using race and ethnicity as part of their admissions to the program. In the end it was judged that the school used a "highly individualized, holistic review of each applicant's file." It was judged this way because the factors were not used in a "mechanical" fashion and was still consistent with the requirement of individualized consideration set in the Univ. Calif v. Bakke. However, the University of Michigan's

undergraduate admission system awarded extra points to minority applicants. This was deemed non-individualized and mechanical because the system was heavily based on race.

The factors that determine your acceptance to an institution should not be based on race, even if it is to meet a quota; it is not ethical in the least. That being said, it should still be subjective because the schools do not want someone that may have a 4.0 GPA and not do anything else, and then the school would not have as many well-rounded students. Good grades are not the prime thing to look for when looking for good people to attend a school, there should be factors like the personal statement, the work experience, the extracurricular activities, or the type of school attended to give admissions a better view of what kind of person they are admitting.

References

Wrangling over Applications. Globe Newspaper Company, 2007. 6 Feb, 2006.

<http://www.boston.com/news/education/higher/articles/2006/02/06/wrangling_over_applications/>

Supreme Court Collection. Cornell University Law School, 2009.

<http://www.law.cornell.edu/supct/cases/topics/tog_discrimination.html>

Ethics Paper

COINS 333

Data Structures

Grade: 70/100

Copyright's Piracy Prevention Plan

Mark Frost

COINS 333

February 9, 2011

In the driver's seat of software development, there are many things to consider once you have completed your project and have produced a polished product for public use. There are issues such as where to sell the product, for how much to sell it for, and how much freedom the customer should have after they purchase your product. The aforementioned should be prioritized in order of importance to your advertising strategy, but setting limits on the amount of freedom that customers have with your product should be priority number one. The biggest reason why controlling some of the ways people use your software is because you would not want someone to steal your ideas or any of your work; this is why, as developer and owner, you should copyright your software. Copyrighting your work will prevent anyone from copying your work and finding financial gain from your intellectual property.

Copyrighting is necessary if you want to keep the individuality of your work and essential if you want to deter piracy of your software. The purpose of copyright law is, "...to promote the Progress of Science and useful Arts, by securing for limited Times to Authors and Inventors the exclusive Right to their respective Writings and Discoveries." (US Constitution, Article 1 section 8). As developer and sole owner there is a certain amount of authority, you should have over your work; the handling and decision making of how to sell the software for instance. Some suggested controls you should have available to you are being able to control how many copies are distributed and what markets it should be available to purchase. In addition, you should be allowed to sell derivatives of your software. If you were to classify parts or plugins for your software under the General Public License (GPL) then that would be the only time modifications could be made by others and it would still be safe from someone profiting from your work. The GPL somewhat relies on the copyright laws to be effective because once software is placed under public domain then it is available for everyone to use but not sell.

Another option you should have is the ability to control who distributes and who sells your product. Allowing only authorized retailers to sell it could help cut down on black market sales that usually end up being sales of a pirated version of the software. You should also retain ownership of the copyright for a period of at least eighty to one hundred years, essentially a lifetime. However, as of the current revision of the copyright laws, the ownership period should be without question.

Under the position of you being the owner, currently, the US Copyright laws are in effect for the lifetime of the author plus 70 years after the author's death.(Copyright Title 3 Ch 17 § 302). The length of time selected is set because it would have been published after January 1, 1978, and anything published before January 1, 1978 would have a time set based on if or when the work was published. The rights that copyright provides to the owner cover a broad spectrum of the expected responsibility of an owner of a product. The following are exclusive rights in copyrighted works as they relate to software copyright; first, they are authorized and have the right, "To reproduce the copyrighted work in copies..." Furthermore, "[The right] to prepare derivative works based upon the copyrighted work." (Copyright Title 17 Ch 1 §106). This means that the owner can produce as many derivatives of the software as they would like. An example of this would be between the PC operating systems of Linux; Ubuntu is a derivative of Debian and Google Chrome OS is a derivative of Ubuntu. They share major parts of the code but differ in enough ways that it can be called under a different name. The final relevant rule for the rights says, "[The right] to distribute copies of the copyrighted work to the public by sale or other transfer of ownership..." (Copyright Title 17 Ch 1 §106). This final rule essentially says that the owner is the only one who can decide who sells the product. These exclusive rights however, do

not compare to the rights that the consumers receive, the rights consumers are allowed to have are partially set by the owner, but majority is set by the Copyright laws.

There are many possibilities for the decision on what activities should be limited for the consumer when it comes to the use of your software. For starters, you should have some control over when and where they can run their software. For instance, if you would like your customer to run it only on a specific version of an operating system or to limit how long the license lasts for the product. On the other side of the equation, the consumer should have options, flexibility when it comes to using your software, especially if it has some proprietary functions apart of it and there are no good substitute programs. For the issue of backup copies, there should be some give and take between the owner and consumer. This issue on the owner's side is if you decide to lock down the software and deny the ability to transfer the license or software to another owner, then you run the risk of piracy becoming a major problem. This would occur due to a number of reasons; one possibility, from the lack of usability, someone who already purchased the software is unable to install it on another computer unless they purchase it again. Software designed similar to this is bound to fail due to the lack of freedom for the end-user, this will cause them to look elsewhere or find illegal copies of your software. However, if you allow them to make backup copies and allow them to use a license multiple times on multiple machines then they will be less likely to pirate the software. This is because the 'value' of the software is higher than if it were locked down due to the flexibility of being able to use at multiple places or times.

Allowing installs on multiple systems or license transfers would reduce piracy and copyright infringement significantly, gain the owner a loyal customer base, and gain repeat buyers for future releases. This method has been implemented in some software of the previous generation, but it has become a much more universally adopted concept for most software

developers. Although piracy is relatively high when compared to ten or twenty years ago, software giants are still breaking profit records because the customers see the software as being worth the money. That, along with a drastic decrease in the chance of being caught for copyright infringement or piracy if using the software for other than its intended purpose. Predator third party companies like the RIAA and MPAA who hunt people that download movies, music, and software will not feel as much of a threat to people now that the newly adopted policies have come about. If copyright infringement should occur, then it should be the responsibility of the owner to seek damages. An organization whose primary goal is to seek maximum damages and set an example out of people they catch is not a good way to go about containing piracy. If anything, it would make it worse because people will find new ways and sources to download from faster than they can catch people; this turns out to be the definition of counterproductive. Therefore, the best way to counteract piracy and cut down on lost profits is for companies to open up their software and allow users to use as they please with limited restrictions.

Works Cited

"Copyright Law: Chapter 3." U.S. Copyright Office. Web. 7 Feb. 2011.

<<http://www.copyright.gov/title17/92chap3.html>>.

"Exclusive Right in Copyrighted Works." *U.S. Copyright Office*. Web. 10 Feb. 2011.

<<http://www.copyright.gov/title17/92chap1.html#106>>.

"What Rights Do Copyright Provide." *CENDI Home Page*. Web. 7 Feb. 2011.

<<http://www.cendi.gov/publications/04-8copyright.html#215>>.

Anagram Maker Design Presentation

COINS 315

Data Structures

Grade: 85/100

Project 5 Design

MARK FROST
COINS 315
NOV. 18, 2010

AnagramMaker (const string &dictionaryFile)

- Create the string array or vector for storing the dictionary
- Using the 'start' and 'end' as char references for each word.
- Read into dictionary file until it reaches the 'start' char for the first letter of the line of the dictionary that's being read. Also set iterator 'dictBegin'. This ignores the the words in dictionary that
- Save dictionary into a vector of strings or a pointer to an array of strings.
- Read until it reaches a word with the start letter just after the 'end' and set iterator 'dictEnd'.

AnagramMaker (const string &dictionaryFile, const string &input)

- Find the highest and lowest letters in the input to determine the the places to start and end in the dictionary. While doing this it will be placed in order as well to help with the search.
- The char variables 'start' and 'end' represent the the highest and lowest letters that the word or phrase can start with.
- For comparison of the chars in the input, will convert individual chars to ints to compare which one is greatest and which is smallest. Set the chars 'start' and 'end' to represent where to start in dictionary.
- Call the constructor that uses just the dictionary.
AnagramMaker(dictionaryFile);

int findAnagrams (string soFar, string input, bool print)

- If print is true then print...
print=1 and return 0 if input is empty
- Start at begining of dictionary 'dictBegin'
- At current word in dictionary, cycle through all letters in input, if letter is in dictionary word then concatenate(sometimes, remove the char that was found, if at beginning or end then use substring.) and call findAnagrams with concatenated word
- If input is <1 then going to return findAnagrams(soFar, input, true) to print what has been found;

Private variables

- struct temp{
- temp(l,c):letter(l),count(c){
- char letter;
- int count;
- }; /*When input organized, places the number of letters available, could make it easier to keep track of how many are left available*/
- char start, end; //keep track of where to start and end in dictionary
- vector<temp> orgInput; //organized input with letter counts, may be used, would need a different values input in the findAnagrams function.
- vector<string> dictionary; //when adding words it could get too big b/c of doubling in size once limit is reached.
- vector<string>::iterator dictEnd, dictBegin;
- string *dictionary; //Other option for storing dictionary, not planning on using yet..

My Online Store proposal

COINS 332

Applied Networking

Grade: 80/100



